Introduction to Transformers with a Numerical Example

Aria Khademi
Independent Researcher

October 2024

1 Abstract

The transformer architecture offered a powerful tool for sequence modeling in machine learning (ML) and with the expansion of practical tools within modern programming languages including Python, the architecture and its variants have been adopted by researchers and practitioners of different domains in various applications. The importance of this architecture and its variants in applications of language modeling cannot be overstated. We provide an easy analytical perspective for a detailed understanding of the architecture. We dive deep into each component of the architecture and explain its mechanism of work as well as the motivation behind using the component. We provide numerical examples of each component and show the algebraic derivations that occur at each step starting from the raw input. The purpose of this article is to provide a detailed introduction of the transformer architecture accompanied by numerical examples. Understanding the mathematical details and the numerical examples will help practitioners and researchers in developing transormers and adopting them more efficiently in their work.

2 Introduction

Transformers can be used to approach a variety of problems, but a main avenue is where a sequence of words is taken and then converted into another sequence of words. For example, take a sentence in English and translate it to French. A seminal paper introduced the transformer architecture in [6] that performs sequence modeling in natural language processing. In this article, we focus on the architecture in the paper mentioned above and explain it in details.

Before transformers were introduced, recurrent neural networks (RNNs) were a standard go-to deep learning method for sequence modeling. However, RNNs suffered two major issues. One was that in RNN calculations, every step t requires values of the previous step $t-1$, those of $t-1$ require values of $t-2$, and so on. Because of this, except for the calculations of every step, nothing can be parallelized in computations. The sequential dependency hinders parallelization capabilities and is hence, slow. Another problem is that because of the sequential nature of the RNN architecture, distant words in a sequence will not fully capture effects and interactions of each other and hence, results will lack context understanding specially for distant words. For example, in the sentence "I go to school every day to learn more and grow as a person," the word "person" is far from the word "school" and RNN will likely miss the relationship and interactions between these two words. The transformer architecture alleviates these issues by providing a mechanism that can parallelize computation more and reduce the computational complexity.

The goal of this article is to explain the details of the transformer architecture and describe its process step by step. We further provide motivations behind architecture components and answer the question: Why is the component used at all? We clarify the concepts with a numerical example that runs through every step of the architecture and show the output that will be obtained from the raw input. For this, we do not train a neural network and use randomly generate examples only for illustration purposes. This article aims to provide a deep understanding of the architecture to help researchers and practitioners develop and expand the methodologies related to the transformer in different applications including those of language modeling.

3 The architecture

Transformers have different architectures in general, with encoder-only models, encoder-decoder models, and decoder-only models. In this article, we focus on the transformer proposed in [6] observable in Figure 1. The architecture contains an encoder and a decoder, each with six identical layers. The purpose of the encoder

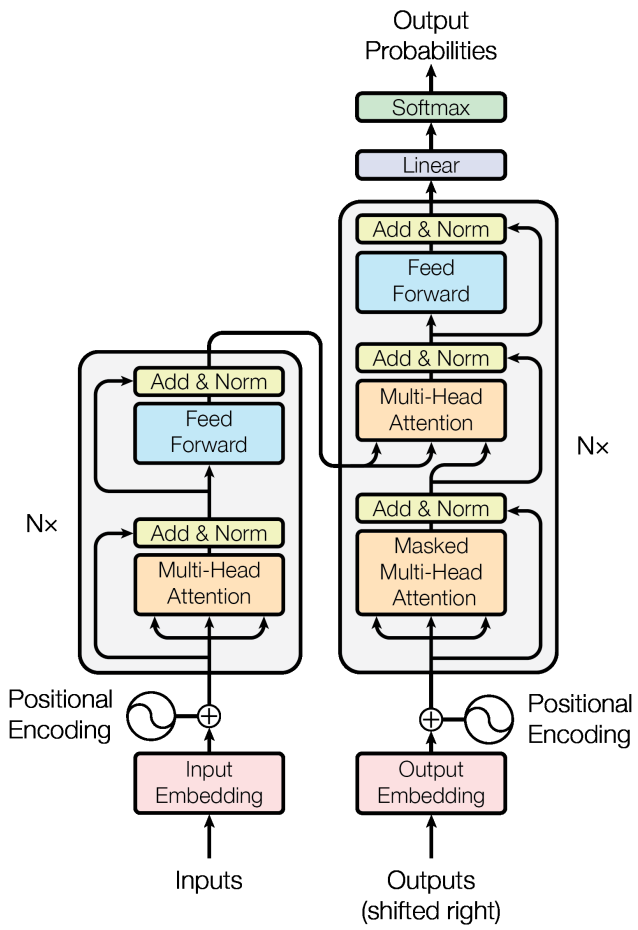

Figure 1: The transformer architecture.

is to generate a contextualized representation for the raw input. By the contextualized representation, we mean an algebraic representation that contains semantic relationships of the words in the sentence and is comprehensible by the computer. In the encoder, every layer contains two sub-layers. Next, the goal of the decoder is, informally speaking, to convert back the algebraic representation (obtained by encoder) into human-readable results, e.g., probabilities of occurrence of words. In the decoder, every layer has three sub-layers. Below we will explain the architecture of a single layer in the encoder.

3.1 Encoder

3.1.1 Input embedding

A transformer takes as input a sequence illustrated here in the form of an unordered set $X = \{x_1, \ldots, x_N\}$, where each x_i is a token. For example, X can be a sentence and each x_i is a word in the sentence such as $X = \{\text{I go to school}\}$. In this, words "I", "go", "to", and "school" are tokens. Since computers do not directly understand human language, we need to convert the tokens into numerical representations. Therefore, each x_i is converted to an integer vector $\mathbf{e_i} = [e_{i1}, \ldots, e_{id}]$. This conversion is called embedding. The literature on word embedding is vast and is out of scope of this article. For the current reading, you can assume that there is an embedding matrix $E \in \mathbb{R}^{|\mathcal{V}| \times d}$, where $\mathcal{V}$ is the size of the vocabulary in the dataset. This means that the embedding matrix acts as a lookup table and we can read the embedding of every token from E by referring to the corresponding row. The embedding matrix is learned during training.

Numerical example. We take the model dimension $d = 4$ for simplicity. Assume that each token in the sentence "I go to school" can be represented by the following 4-dimensional embeddings where the numbers are random and are only for illustration purposes:

- "I": $\begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \end{pmatrix}$
- "go": $\begin{pmatrix} 0.5 & 0.6 & 0.7 & 0.8 \end{pmatrix}$
- "to": $\begin{pmatrix} 0.9 & 0.1 & 0.2 & 0.3 \end{pmatrix}$
- "school": $\begin{pmatrix} 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix}$

The sentence "I go to school" is then represented by the following matrix, which is the result of input embedding, and where each row corresponds to a word's embedding:

$$A = \begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix}$$

3.1.2 Positional encoding

After input embedding is completed, positional encoding is conducted on the input embedding for the following reason. Recall that traditionally RNNs operate on the tokens sequentially where every step t requires step $t-1$. Hence, the RNN captures the positioning or ordering of the tokens, but it is slow. The transformer architecture speeds up the training and prediction through parallelizing processing of tokens. However, parallelization will lose the ordering or the position of the tokens within the sentence and hence, fails to capture the context. To alleviate this, a numerical vector is computed per each token and is added to

its embedded vector. This is called positional encoding. There is literature on positional encoding [1, 5, 2] which is outside the scope of this article, but [6] has suggested to add the positional encoding according to the following:

$$PE_{\text{pos},2i} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right) \tag{1}$$

$$PE_{\text{pos},2i+1} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d}}}\right) \tag{2}$$

In the above, pos refers to the position or index of the token in the sentence, e.g., 0 for the first token, $0 \leq i < m/2$ (m being the length of the embedding vector) is the dimension within the embedding vector $\mathbf{e_i}$ and takes a sinusoidal function for even indices and cosinusoidal function for odd indices, and d is the model dimension. The result is a vector $\mathbf{p_i}$ of size d.

To account for the position of each x_i in the sentence, we obtain a representation $\mathbf{z_i} = \mathbf{e_i} + \mathbf{p_i}$, for each token, where $\mathbf{p_i}$ is a positional encoding vector and $\mathbf{e_i}$ is the embedding vector. For example, assume that the input sentence is "I go to school". X is the set {I, go, to, school}, where $x_3 = \textit{school}$ and so on. We first build a representation vector $\mathbf{e_3}$ and then add the vector $\mathbf{p_3}$ to it to obtain $\mathbf{z_3}$ for "school" and so on. Then, $\mathbf{z_3}$ will be used as the numerical representation of the word "school" and is fed to the next step of the architecture.

Numerical example. Since we have 4 dimensions, we compute the encoding for indices 0, 1, 2, and 3. The positional encoding for each word is calculated as follows:

For position 0:

$$PE_{0,0} = \sin\left(\frac{0}{10000^{0/4}}\right) = 0, \quad PE_{0,1} = \cos\left(\frac{0}{10000^{0/4}}\right) = 1$$
$$PE_{0,2} = \sin\left(\frac{0}{10000^{2/4}}\right) = 0, \quad PE_{0,3} = \cos\left(\frac{0}{10000^{2/4}}\right) = 1$$

Therefore, the resulting representation for position 0 is:

$$PE(0) = \begin{pmatrix} 0 & 1 & 0 & 1 \end{pmatrix}$$

Similarly, for position 1:

$$PE_{1,0} = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1), \quad PE_{1,1} = \cos\left(\frac{1}{10000^{0/4}}\right) = \cos(1)$$
$$PE_{1,2} = \sin\left(\frac{1}{10000^{2/4}}\right), \quad PE_{1,3} = \cos\left(\frac{1}{10000^{2/4}}\right)$$

Therefore, the resulting representation is:

$$PE(1) = \begin{pmatrix} \sin(1) & \cos(1) & \sin\left(\frac{1}{10000^{0.5}}\right) & \cos\left(\frac{1}{10000^{0.5}}\right) \end{pmatrix}$$

For the remaining indices, we omit the details and show the result.

For position 2:

$$PE(2) = \begin{pmatrix} \sin(2) & \cos(2) & \sin\left(\frac{2}{10000^{0.5}}\right) & \cos\left(\frac{2}{10000^{0.5}}\right) \end{pmatrix}$$

For position 3:

$$PE(3) = \begin{pmatrix} \sin(3) & \cos(3) & \sin\left(\frac{3}{10000^{0.5}}\right) & \cos\left(\frac{3}{10000^{0.5}}\right) \end{pmatrix}$$

Now, adding the positional encodings $\mathbf{p_i}$ to the word embeddings $\mathbf{e_i}$ obtained in the previous section and stacking them up together to form a matrix, we will obtain:

$$Z = \begin{pmatrix} 0.1+0 & 0.2+1 & 0.3+0 & 0.4+1 \\ 0.5+\sin(1) & 0.6+\cos(1) & 0.7+\sin\left(\frac{1}{10000^{0.5}}\right) & 0.8+\cos\left(\frac{1}{10000^{0.5}}\right) \\ 0.9+\sin(2) & 0.1+\cos(2) & 0.2+\sin\left(\frac{2}{10000^{0.5}}\right) & 0.3+\cos\left(\frac{2}{10000^{0.5}}\right) \\ 0.4+\sin(3) & 0.5+\cos(3) & 0.6+\sin\left(\frac{3}{10000^{0.5}}\right) & 0.7+\cos\left(\frac{3}{10000^{0.5}}\right) \end{pmatrix}$$

$$= \begin{pmatrix} 0.1 & 1.2 & 0.3 & 1.4 \\ 1.34 & 1.14 & 0.71 & 1.80 \\ 1.81 & -0.32 & 0.22 & 1.30 \\ 0.54 & -0.49 & 0.63 & 1.70 \end{pmatrix} = \begin{pmatrix} \mathbf{z}_{\mathrm{I}} \\ \mathbf{z}_{\mathrm{go}} \\ \mathbf{z}_{\mathrm{to}} \\ \mathbf{z}_{\mathrm{school}} \end{pmatrix}$$

3.1.3 Attention mechanism

We first provide an intuition toward the attention mechanism and then, proceed with explaining it in detail below. The attention mechanism requires a dictionary with keys and values for all the tokens in the vocabulary, i.e., for each token, or key, we have a value. Then, assume that we observe a query, e.g., in the form of a token, and search for the key that matches the query. We are not looking for an exact match, and rather, will obtain the degree to which the query matches with, or "attends" to each key. This is called soft attending. After computing the degrees of matching, attention is intuitively captured by a weighted sum of the attendances. Note that while attention is by definition computed over all tokens, in practice, the computation is done using matrix multiplications that we will explain in Section 3.1.4.

We now proceed by providing a more formal definition. Assume that the model has learned weight matrices $W_Q \in \mathbb{R}^{d \times d_k}$, $W_K \in \mathbb{R}^{d \times d_k}$, $W_V \in \mathbb{R}^{d \times d_v}$ during training, where subscripts Q, K, and V refer to query, key, and value, respectively. For an input x_i, we obtain an embedded vector $\mathbf{z_i}$ of size d. Then, a query, a key, and a value are computed according to $\mathbf{q_i} = \mathbf{z_i} W_Q$, $\mathbf{k_i} = \mathbf{z_i} W_K$, $\mathbf{v_i} = \mathbf{z_i} W_V$.[1] We have now computed $\mathbf{q_i}$, $\mathbf{k_i}$, and $\mathbf{v_i}$ for all tokens x_i, where $\mathbf{q_i}$ and $\mathbf{k_i}$ are all in the shape $1 \times d_k$ and $\mathbf{v_i}$ have the shape $1 \times d_v$. Then, for each pair of tokens x_i and x_j within the same sequence, we compute a score:

$$s_{i,j} = \frac{\mathbf{q_i} . \mathbf{k_j}}{\sqrt{d_k}}, \tag{3}$$

where $\sqrt{d_k}$ is a scaling factor that prevents the dot product from growing too large in magnitude. We explain later in Section 3.1.6 that the vectors are normalized to mean 0 and variance 1. However, since variance of sum equals sum of variances (assume independence), in the dot products of computing the scores which are sum of element-wise multiplications, variance will be d_k (because $1 \times d_k = d_k$) and hence, standard deviation is $\sqrt{d}$. We know that the growth magnitude of dot products is measured by the standard deviation, which is $\sqrt{d}$. Without scaling, the dot product will grow large in magnitude and can saturate the Softmax function or explode gradients.

The scores are next fed to a Softmax function as follows:

[1] Note that more commonly in linear algebra of neural networks, we write this as Wx with W as the weight matrix and x as the input vector, but we kept the dimensions consistent with the original transformer paper.

$$\alpha_{i,j} = \frac{e^{s(i,j)}}{\Sigma_{j=1}^{n} e^{(s(i,j))}}. \tag{4}$$

Finally, the output for any token x_i is computed as the following weighted sum:

$$a_i = \Sigma_{j=1}^{n} \alpha_{ij}.\mathbf{v_j}. \tag{5}$$

The dot product in Equation 5 is called "attention". In simple words, attention is defined over the query, key, values, and the input token. The attention is computed over the tokens in the same sequence as opposed to other cross-attention mechanisms that compute these across different sequences. Because of the preceding, the attention mechanism in [6] explained above is called "self-attention." The purpose of self-attention is to capture the contextual relationship between tokens in the same sequence.

Numerical example. Recall that the weight matrices are learned during training and for this example, we assume that they are the following matrices where the matrix elements are chosen at random for illustration purposes:

$$W_Q = \begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix}, W_K = \begin{pmatrix} 0.2 & 0.1 & 0.4 & 0.3 \\ 0.3 & 0.7 & 0.6 & 0.5 \\ 0.8 & 0.2 & 0.1 & 0.9 \\ 0.5 & 0.4 & 0.3 & 0.2 \end{pmatrix}, W_V = \begin{pmatrix} 0.3 & 0.6 & 0.1 & 0.4 \\ 0.5 & 0.7 & 0.8 & 0.2 \\ 0.9 & 0.1 & 0.5 & 0.3 \\ 0.2 & 0.3 & 0.4 & 0.6 \end{pmatrix}$$

We move forward to compute attention for the word "school". For that, we compute the query, key, and values for this word as follows.

$$\begin{aligned} \boldsymbol{q}_{\text{school}} &= \begin{pmatrix} 0.54 & -0.49 & 0.63 & 1.70 \end{pmatrix} \times \begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix} \\ &= \begin{pmatrix} 1.056 & 0.727 & 0.965 & 1.203 \end{pmatrix} \end{aligned}$$

$$\begin{aligned} \boldsymbol{k}_{\text{school}} &= \begin{pmatrix} 0.54 & -0.49 & 0.63 & 1.70 \end{pmatrix} \times \begin{pmatrix} 0.2 & 0.1 & 0.4 & 0.3 \\ 0.3 & 0.7 & 0.6 & 0.5 \\ 0.8 & 0.2 & 0.1 & 0.9 \\ 0.5 & 0.4 & 0.3 & 0.2 \end{pmatrix} \\ &= \begin{pmatrix} 1.315 & 0.517 & 0.495 & 0.824 \end{pmatrix} \end{aligned}$$

$$\begin{aligned} \boldsymbol{v}_{\text{school}} &= \begin{pmatrix} 0.54 & -0.49 & 0.63 & 1.70 \end{pmatrix} \times \begin{pmatrix} 0.3 & 0.6 & 0.1 & 0.4 \\ 0.5 & 0.7 & 0.8 & 0.2 \\ 0.9 & 0.1 & 0.5 & 0.3 \\ 0.2 & 0.3 & 0.4 & 0.6 \end{pmatrix} \\ &= \begin{pmatrix} 0.824 & 0.554 & 0.657 & 1.327 \end{pmatrix} \end{aligned}$$

To compute attention for the word school, we need to compute the key, and value for each other word.

For "I":

$$\boldsymbol{k}_{\mathrm{I}} = \begin{pmatrix} 0.213 & 0.387 & 0.612 & 0.889 \end{pmatrix}, \boldsymbol{v}_{\mathrm{I}} = \begin{pmatrix} 0.375 & 0.499 & 0.237 & 0.673 \end{pmatrix}$$

For "go":

$$\boldsymbol{k}_{\mathrm{go}} = \begin{pmatrix} 0.749 & 0.301 & 0.547 & 0.812 \end{pmatrix}, \boldsymbol{v}_{\mathrm{go}} = \begin{pmatrix} 0.467 & 0.659 & 0.391 & 0.883 \end{pmatrix}$$

For "to":

$$\boldsymbol{k}_{\mathrm{to}} = \begin{pmatrix} 0.452 & 0.675 & 0.854 & 0.274 \end{pmatrix}, \boldsymbol{v}_{\mathrm{to}} = \begin{pmatrix} 0.235 & 0.678 & 0.823 & 0.453 \end{pmatrix}$$

For "school":

$$\begin{aligned} \boldsymbol{q}_{\mathrm{school}} &= \begin{pmatrix} 1.056 & 0.727 & 0.965 & 1.203 \end{pmatrix}, \\ \boldsymbol{k}_{\mathrm{school}} &= \begin{pmatrix} 1.315 & 0.517 & 0.495 & 0.824 \end{pmatrix}, \\ \boldsymbol{v}_{\mathrm{school}} &= \begin{pmatrix} 0.824 & 0.554 & 0.657 & 1.327 \end{pmatrix} \end{aligned}$$

Now compute the dot product of $\boldsymbol{q}_{\mathrm{school}}$ with the key vectors of all the words in the sentence.

$$\begin{aligned} \boldsymbol{q}_{\mathrm{school}} \cdot \boldsymbol{k}_{\mathrm{I}} &= (1.056 \times 0.213) + (0.727 \times 0.387) + (0.965 \times 0.612) + (1.203 \times 0.889) \\ &= 0.224928 + 0.281649 + 0.59058 + 1.069467 = 2.166624 \end{aligned}$$

$$\begin{aligned} \boldsymbol{q}_{\mathrm{school}} \cdot \boldsymbol{k}_{\mathrm{go}} &= (1.056 \times 0.749) + (0.727 \times 0.301) + (0.965 \times 0.547) + (1.203 \times 0.812) \\ &= 0.791944 + 0.218727 + 0.527255 + 0.977436 = 2.515362 \end{aligned}$$

$$\begin{aligned} \boldsymbol{q}_{\mathrm{school}} \cdot \boldsymbol{k}_{\mathrm{to}} &= (1.056 \times 0.452) + (0.727 \times 0.675) + (0.965 \times 0.854) + (1.203 \times 0.274) \\ &= 0.477312 + 0.490725 + 0.82331 + 0.329622 = 2.121969 \end{aligned}$$

$$\begin{aligned} \boldsymbol{q}_{\mathrm{school}} \cdot \boldsymbol{k}_{\mathrm{school}} &= (1.056 \times 1.315) + (0.727 \times 0.517) + (0.965 \times 0.495) + (1.203 \times 0.824) \\ &= 1.38864 + 0.375959 + 0.477675 + 0.991272 = 3.233546 \end{aligned}$$

Now, scale these dot products by $\frac{1}{\sqrt{d_k}}$ where $d_k = 4$ (dimension of vectors):

$$s_{I,school} = \frac{2.166624}{2} = 1.083312, s_{go,school} = \frac{2.515362}{2} = 1.257681$$

$$s_{to,school} = \frac{2.121969}{2} = 1.060985, s_{school,school} = \frac{3.233546}{2} = 1.616773$$

Now apply the Softmax function on these scaled values:

$$e^{1.083312} = 2.954572, e^{1.257681} = 3.517688, e^{1.060985} = 2.889919, e^{1.616773} = 5.036707$$

The sum of these exponentials is:

$$\sum_j e^{x_j} = 2.954572 + 3.517688 + 2.889919 + 5.036707 = 14.398886$$

Now, compute the Softmax values:

$$\alpha_{I,school} = \frac{2.954572}{14.398886} = 0.2052, \alpha_{go,school} = \frac{3.517688}{14.398886} = 0.2443$$
$$\alpha_{to,school} = \frac{2.889919}{14.398886} = 0.2007, \alpha_{school,school} = \frac{5.036707}{14.398886} = 0.3498$$

Finally, compute the weighted sum of the value vectors:

$$\begin{aligned} a_{school} &= 0.2052 \cdot \boldsymbol{v}_{\text{I}} + 0.2443 \cdot \boldsymbol{v}_{\text{go}} + 0.2007 \cdot \boldsymbol{v}_{\text{to}} + 0.3498 \cdot \boldsymbol{v}_{\text{school}} \\ &= 0.2052 \cdot \begin{pmatrix} 0.375 & 0.499 & 0.237 & 0.673 \end{pmatrix} \\ &+ 0.2443 \cdot \begin{pmatrix} 0.467 & 0.659 & 0.391 & 0.883 \end{pmatrix} \\ &+ 0.2007 \cdot \begin{pmatrix} 0.235 & 0.678 & 0.823 & 0.453 \end{pmatrix} \\ &+ 0.3498 \cdot \begin{pmatrix} 0.824 & 0.554 & 0.657 & 1.327 \end{pmatrix} \end{aligned}$$

Final attention output for the word "school":

$$a_{school} = \begin{pmatrix} 0.52664 & 0.59352 & 0.53919 & 0.90895 \end{pmatrix}$$

3.1.4 Attention in matrix form

While self-attention is defined over tokens, in practice, values of $\mathbf{q_i}$, $\mathbf{k_i}$, and $\mathbf{v_i}$ are stacked together to form matrices. In stacking these, assume that we have N tokens in our input and compute $\mathbf{q_i}$, $\mathbf{k_i}$, and $\mathbf{v_i}$ for all of them. We then stack them together and form matrices $Q \in \mathbb{R}^{N \times d_k}$, $K \in \mathbb{R}^{N \times d_k}$, and $V \in \mathbb{R}^{N \times d_v}$. Then, the following will be computed:

$$\text{ATN} = \textsc{Self-Attention}(Q, K, V) = \textsc{Softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{6}$$

The output of attention will then have the shape $N \times d_v$.

Numerical example. In the previous section, we provided an example of computing attention for the word "school" to illustrate the details, but as mentioned above, in practice, attentions are computed in matrix form. Here, we proceed with showing the calculations in matrix form and abandon showing the calculations of each vector separately. The matrices Q, K, and V can be computed as follows:

$$Q = Z \times W_Q = \begin{pmatrix} 0.1 & 1.2 & 0.3 & 1.4 \\ 1.34 & 1.14 & 0.71 & 1.80 \\ 1.81 & -0.32 & 0.22 & 1.30 \\ 0.54 & -0.49 & 0.63 & 1.70 \end{pmatrix} \times \begin{pmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 & 0.7 \end{pmatrix}$$

$$= \begin{pmatrix} 1.44 & 1.47 & 1.77 & 2.07 \\ 2.063 & 1.923 & 2.422 & 2.921 \\ 0.739 & 0.842 & 1.143 & 1.444 \\ 1.056 & 0.727 & 0.965 & 1.203 \end{pmatrix} = \begin{pmatrix} \mathbf{q}_{\mathrm{I}} \\ \mathbf{q}_{\mathrm{go}} \\ \mathbf{q}_{\mathrm{to}} \\ \mathbf{q}_{\mathrm{school}} \end{pmatrix}$$

$$K = Z \times W_K = \begin{pmatrix} 0.1 & 1.2 & 0.3 & 1.4 \\ 1.34 & 1.14 & 0.71 & 1.80 \\ 1.81 & -0.32 & 0.22 & 1.30 \\ 0.54 & -0.49 & 0.63 & 1.70 \end{pmatrix} \times \begin{pmatrix} 0.2 & 0.1 & 0.4 & 0.3 \\ 0.3 & 0.7 & 0.6 & 0.5 \\ 0.8 & 0.2 & 0.1 & 0.9 \\ 0.5 & 0.4 & 0.3 & 0.2 \end{pmatrix}$$

$$= \begin{pmatrix} 1.32 & 1.47 & 1.21 & 1.18 \\ 2.078 & 1.794 & 1.831 & 1.971 \\ 1.092 & 0.521 & 0.944 & 0.841 \\ 1.315 & 0.517 & 0.495 & 0.824 \end{pmatrix} = \begin{pmatrix} \mathbf{k}_{\mathrm{I}} \\ \mathbf{k}_{\mathrm{go}} \\ \mathbf{k}_{\mathrm{to}} \\ \mathbf{k}_{\mathrm{school}} \end{pmatrix}$$

$$V = Z \times W_V = \begin{pmatrix} 0.1 & 1.2 & 0.3 & 1.4 \\ 1.34 & 1.14 & 0.71 & 1.80 \\ 1.81 & -0.32 & 0.22 & 1.30 \\ 0.54 & -0.49 & 0.63 & 1.70 \end{pmatrix} \times \begin{pmatrix} 0.3 & 0.6 & 0.1 & 0.4 \\ 0.5 & 0.7 & 0.8 & 0.2 \\ 0.9 & 0.1 & 0.5 & 0.3 \\ 0.2 & 0.3 & 0.4 & 0.6 \end{pmatrix}$$

$$= \begin{pmatrix} 1.18 & 1.35 & 1.68 & 1.21 \\ 1.971 & 2.213 & 2.121 & 2.057 \\ 0.841 & 1.274 & 0.555 & 1.506 \\ 0.824 & 0.554 & 0.657 & 1.327 \end{pmatrix} = \begin{pmatrix} \mathbf{v}_{\mathrm{I}} \\ \mathbf{v}_{\mathrm{go}} \\ \mathbf{v}_{\mathrm{to}} \\ \mathbf{v}_{\mathrm{school}} \end{pmatrix}$$

We now calculate self-attention according to Equation 6.

$$QK^T = \begin{pmatrix} 9.62073 & 6.75094 & 7.1295 & 7.78012 \\ 13.225993 & 9.259892 & 9.741206 & 10.665669 \\ 5.872132 & 3.940849 & 4.230884 & 4.681901 \\ 5.545841 & 3.980054 & 4.112582 & 4.482294 \end{pmatrix}$$

$$\mathrm{Softmax}\left(\frac{QK^T}{2}\right) = \begin{pmatrix} 0.518 & 0.125 & 0.150 & 0.207 \\ 0.628 & 0.087 & 0.110 & 0.175 \\ 0.416 & 0.158 & 0.196 & 0.230 \\ 0.394 & 0.180 & 0.192 & 0.233 \end{pmatrix}$$

Hence, the attention matrix will be:

$$\textsc{atn}^{(1)} = \mathrm{Softmax}\left(\frac{QK^T}{2}\right) V \approx \begin{pmatrix} 1.1548 & 1.2816 & 1.3542 & 1.3840 \\ 1.1190 & 1.2782 & 1.4161 & 1.3377 \\ 1.1553 & 1.2884 & 1.2935 & 1.4340 \\ 1.1732 & 1.3039 & 1.3041 & 1.4451 \end{pmatrix}$$

The superscript (1) in $\textsc{atn}^{(1)}$ is used to refer to this as the first head of the multi-head attention. The multi-head structure is explained below.

3.1.5 Multi-head attention

The transformer architecture includes multi-head attentions meaning that the preceding calculations of Equation 6 are done multiple times in parallel, each time with different learned weight matrices $W_Q^{head_h}$, $W_K^{head_h}$, $W_V^{head_h}$. The number of attention heads is arbitrary and is set to 6 in [6]. The learned weight matrices $W_Q^{head_h}$, $W_K^{head_h}$, $W_V^{head_h}$ are different in each head and are learned during training. The reason for having multiple heads compared to just one head is that each head captures specific semantics in the sequence. Recall that the output of each attention head has the shape $N \times d_v$. Once all attentions are computed in heads, their outputs are concatenated as follows:

$$\text{ATN}^{\text{concat}} = \text{Concat}(\text{ATN}^{(1)}, \text{ATN}^{(2)}, \ldots, \text{ATN}^{(h)}). \tag{7}$$

The above will give a result with the shape $\text{ATN}^{\text{concat}} \in N \times hd_v$. The total attention is then projected onto another space using the linear transformation below:

$$O^{mh} = \text{ATN}^{\text{concat}} W^O, \tag{8}$$

where $W^O \in \mathbb{R}^{hdv \times d}$ is another weight matrix learned during training whose job is to revert the concatenated output back to the original input dimensions. In this, note that the computations of every head is done in a lower dimensional space for faster computation and we need to revert the matrices back to the original space. It can be observed that output of the multi-head attention has the shape $N \times d$.

Numerical example. For simplicity of the example, imagine we have two attention heads. In the first one, the attention matrix has been computed in the previous section. Call it $\text{ATN}^{(1)}$. For the second head, we have generated a random attention matrix. Call it $\text{ATN}^{(2)}$.

$$\text{ATN}^{(2)} \approx \begin{pmatrix} 1.0234 & 1.4876 & 1.7523 & 1.6432 \\ 1.1987 & 1.3095 & 1.4058 & 1.2389 \\ 0.9345 & 1.1842 & 1.3976 & 1.5210 \\ 1.3456 & 1.2703 & 1.4389 & 1.5824 \end{pmatrix}$$

$$\text{ATN}^{\text{concat}} = \text{Concat}(\text{ATN}^{(1)}, \text{ATN}^{(2)})$$

$$= \begin{pmatrix} 1.1548 & 1.2816 & 1.3542 & 1.3840 & 1.0234 & 1.4876 & 1.7523 & 1.6432 \\ 1.1190 & 1.2782 & 1.4161 & 1.3377 & 1.1987 & 1.3095 & 1.4058 & 1.2389 \\ 1.1553 & 1.2884 & 1.2935 & 1.4340 & 0.9345 & 1.1842 & 1.3976 & 1.5210 \\ 1.1732 & 1.3039 & 1.3041 & 1.4451 & 1.3456 & 1.2703 & 1.4389 & 1.5824 \end{pmatrix}$$

Now, in the following, we have generated a random matrix, W^O, with appropriate dimensions. In practice, this matrix is learned during training.

$$W^O = \begin{pmatrix} 0.1 & 0.4 & 0.7 & 0.2 \\ 0.5 & 0.2 & 0.9 & 0.6 \\ 0.3 & 0.8 & 0.1 & 0.4 \\ 0.9 & 0.3 & 0.6 & 0.5 \\ 0.2 & 0.7 & 0.8 & 0.1 \\ 0.6 & 0.4 & 0.3 & 0.2 \\ 0.1 & 0.5 & 0.2 & 0.9 \\ 0.8 & 0.6 & 0.4 & 0.3 \end{pmatrix}$$

After generating the random matrix, W^O, we can compute O^{mh} as follows.

$$O^{mh} = \text{ATN}^{\text{concat}} W^O \approx \begin{pmatrix} 4.9999 & 4.0412 & 5.0548 & 4.5789 \\ 3.5369 & 4.5385 & 5.0069 & 3.5540 \\ 4.6906 & 4.0801 & 5.0485 & 3.1040 \\ 4.8302 & 4.4255 & 4.5940 & 3.1230 \end{pmatrix}$$

3.1.6 Add and normalize

In the next step, the input is added to the output of the multi-head attention. This is known as residual connection introduced in [3]. Residual connections are particularly used in computer vision and help deep learning models in optimization by smoothing the path toward the global optimum. In the transformer architecture, residual connection is used as follows:

$$O^{\text{res}} = O^{mh} + Z, \tag{9}$$

where $Z \in \mathbb{R}^{N \times d}$ is the matrix form of N embedded tokens stacked together. Next, the output is normalized via the layer normalization approach of [4]:

$$O^{\text{norm}} = \text{LayerNorm}(O^{\text{res}}). \tag{10}$$

Assume that a specific token is represented using the kth row of matrix O^{res}. Normalization takes a vector as input and outputs a vector. It is applied to the kth row of the matrix O^{res}, within each layer, as follows. Denote the row with a vector $\mathbf{v} = \{v_1, \ldots, v_d\}$, where d is the model dimension. Compute its mean and standard deviation by:

$$\mu = \frac{1}{d} \sum_{i=1}^{d} v_i, \tag{11}$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^{d} (v_i - \mu)^2}. \tag{12}$$

Then, normalize as follows:

$$\hat{\mathbf{v}}_i = \frac{\mathbf{v}_i - \mu}{\sigma} \cdot \gamma + \beta, \tag{13}$$

where γ and β are values learned during training and are used for scaling and shifting.

Numerical example.

$$O^{\text{res}} = O^{mh} + Z =$$

$$\begin{pmatrix} 4.9999 & 4.0412 & 5.0548 & 4.5789 \\ 3.5369 & 4.5385 & 5.0069 & 3.5540 \\ 4.6906 & 4.0801 & 5.0485 & 3.1040 \\ 4.8302 & 4.4255 & 4.5940 & 3.1230 \end{pmatrix} + \begin{pmatrix} 0.1 & 1.2 & 0.3 & 1.4 \\ 1.34 & 1.14 & 0.71 & 1.80 \\ 1.81 & -0.32 & 0.22 & 1.30 \\ 0.54 & -0.49 & 0.63 & 1.70 \end{pmatrix}$$

$$= \begin{pmatrix} 5.0999 & 5.2412 & 5.3548 & 5.9789 \\ 4.8769 & 5.6785 & 5.7169 & 5.3540 \\ 6.5006 & 3.7601 & 5.2685 & 4.4040 \\ 5.3702 & 3.9355 & 5.2240 & 4.8230 \end{pmatrix}$$

We normalize by row assuming that $\gamma = 1$ and $\beta = 0$ as follows:

$$O^{\text{norm}} = \begin{pmatrix} -0.9494 & -0.5286 & -0.1903 & 1.6683 \\ -1.5730 & 0.8075 & 0.9216 & -0.1561 \\ 1.4780 & -1.1915 & 0.2778 & -0.5643 \\ 0.9529 & -1.6167 & 0.6910 & -0.0272 \end{pmatrix}$$

3.1.7 Feed forward neural network

So far, the architecture contains only weighted averages and is missing the nonlinear properties of neural networks. A feed forward neural network is used at this point to add non-linearity to the structure instead of having only weighted averages. The feed forward neural network contains two linear transformation and a ReLU (rectified linear unit) activation function in between them. Input to the feed forward neural network is a row k from the matrix O^{norm}, denoted here as O_k^{norm}. Note that the feed forward network processes tokens independently.

$$O_k^{\text{ffn}} = \text{ReLU}(O_k^{\text{norm}} W_1 + \mathbf{b_1}) W_2 + \mathbf{b_2}, \tag{14}$$

where W_1, W_2, $\mathbf{b_1}$, and $\mathbf{b_2}$ are weight matrices and biases learned during training, and $\text{ReLU}(\cdot)$ is a function $f(x) = \max(0, x)$. Shape of the matrix O^{ffn} will be $N \times d$.

Numerical example. We will next run the matrix O^{norm} through the feed forward neural network and assume that we have access to the learned weight matrices. For this, we have generated random matrices for illustration and have not trained a neural network. Assume that the parameters of the neural network are the following:

$$W_1 = \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \\ 0.1 & 0.2 & 0.1 \end{pmatrix}$$

$$W_2 = \begin{pmatrix} 0.3 & 0.2 & 0.1 & 0.4 \\ 0.5 & 0.4 & 0.3 & 0.2 \\ 0.2 & 0.1 & 0.3 & 0.5 \end{pmatrix}$$

$$\mathbf{b_1} = \begin{pmatrix} 0.1 & 0.2 & 0.3 \end{pmatrix}$$

$$\mathbf{b_2} = \begin{pmatrix} 0.1 & 0.2 & 0.1 & 0.1 \end{pmatrix}$$

For $O_1^{\text{norm}} = \begin{pmatrix} -0.9494 & -0.5286 & -0.1903 & 1.6683 \end{pmatrix}$:

$$O_1^{\text{norm}} W_1 = \begin{pmatrix} -0.9494 & -0.5286 & -0.1903 & 1.6683 \end{pmatrix} \begin{pmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \\ 0.1 & 0.2 & 0.1 \end{pmatrix}$$

$$= \begin{pmatrix} 0.3031 & -0.4826 & -0.9938 \end{pmatrix}$$

Add $\mathbf{b_1}$:

$$O_1^{\text{norm}} W_1 + \mathbf{b_1} = \begin{pmatrix} 0.3031 & -0.4826 & -0.9938 \end{pmatrix} + \begin{pmatrix} 0.1 & 0.2 & 0.3 \end{pmatrix}$$

$$= \begin{pmatrix} 0.4031 & -0.2826 & -0.6938 \end{pmatrix}$$

Apply ReLU:

$$\text{ReLU}(O_1^{\text{norm}} W_1 + \mathbf{b_1}) = \begin{pmatrix} 0.4031 & 0 & 0 \end{pmatrix}$$

Compute $\text{ReLU}(O_1 W_1 + \mathbf{b_1}) W_2$:

$$\begin{pmatrix} 0.4031 & 0 & 0 \end{pmatrix} W_2 = \begin{pmatrix} 0.1209 & 0.0806 & 0.0403 & 0.1612 \end{pmatrix}$$

Add $\mathbf{b_2}$:

$$O_1^{\text{ffn}} = \begin{pmatrix} 0.1209 & 0.0806 & 0.0403 & 0.1612 \end{pmatrix} + \begin{pmatrix} 0.1 & 0.2 & 0.1 & 0.1 \end{pmatrix}$$

$$= \begin{pmatrix} 0.2209 & 0.2806 & 0.1403 & 0.2612 \end{pmatrix}$$

Applying similar calculations to other rows of the O^{norm} matrix results in:

$$O^{\text{ffn}} = \begin{pmatrix} 0.2209 & 0.6150 & 0.2182 & 0.17976 \\ 0.2806 & 0.5798 & 0.2588 & 0.23988 \\ 0.1403 & 0.4078 & 0.3242 & 0.21964 \\ 0.2612 & 0.7413 & 0.3540 & 0.2994 \end{pmatrix}$$

3.1.8 Add and normalize

The output of the feed forward network is added to its input through residual connection and normalized with layer normalization one more time.

$$O^{\text{res,ffn}} = O^{\text{ffn}} + O^{\text{norm}} \tag{15}$$

$$O^{\text{norm,ffn}} = \text{LayerNorm}(O^{\text{res,ffn}}) \tag{16}$$

The above $O^{\text{norm,ffn}}$ is the output of the encoder, which we will call O^{enc}.

Numerical example.

$$O^{\text{res,ffn}} = O^{\text{ffn}} + O^{\text{norm}} = \begin{pmatrix} -0.7285 & 0.0864 & 0.0279 & 1.84806 \\ -1.2924 & 1.3873 & 1.1804 & 0.08378 \\ 1.6183 & -0.7837 & 0.6020 & -0.34466 \\ 1.2141 & -0.8754 & 1.0450 & 0.2722 \end{pmatrix}$$

We now apply row-wise normalization to this matrix as described before.

$$O^{\text{norm,ffn}} = \begin{pmatrix} -1.1852 & -0.2535 & -0.3204 & 1.7589 \\ -1.0332 & 0.7732 & 0.6104 & -0.0136 \\ 1.0070 & -1.0594 & 0.7373 & -0.6850 \\ 1.3370 & -1.0716 & 0.9600 & -0.2254 \end{pmatrix} = O^{\text{enc}}$$

3.2 Decoder

The architecture of the decoder is similar to that of the encoder, but has an extra block called the masked multi-head attention explained below in Section 3.2.2. We abstain from reiterating the similar blocks, but describe the architecture in general. The input of the decoder is embedded and then positional encoding is added. The result is fed into a masked multi-head attention block followed by residual connection and layer normalization. Next, this output along with the output of the encoder are fed into the non-masked multi-head attention block followed by residual connection and layer normalization. After that, a feed forward neural network is used to adopt non-linear properties. This is then followed by residual connection and layer normalization. Next, a linear transformation and a Softmax function are applied, which produce the final outputs, which are probabilities of tokens. The calculations are similar to the encoder, so we do not reiterate the numerical example.

3.2.1 Decoder input

Input to the decoder varies by application, but is generally the desired output shifted right. For example, in translation, input to the encoder is a sentence in one language, and the input to the decoder is the translation of that sequence in another language. For instance, if the input to the encoder is "I go to school" and the desired task is translation to French, then in training, "Je vais à l'école" shifted right is the input to the decoder. This means that considering the sentence in the form of ["`<start>`", "Je", "vais", "à", "l'", "école"], where `<start>` is the starting token, then the array ["Je", "vais", "à", "l'", "école"] is the input to the decoder. Note that the right shift happens in training but not in inference or generation. During inference, the input is the starting token `<start>`, and the model generates the next tokens one by one.

Similar to the encoder, the input is embedded and positional encoding is applied with a minor difference. The embedding matrix is different from that of the encoder because the type of the sequence is different in nature although the positional encoding adopts the same formula used in the encoder.

3.2.2 Masked multi-head attention

In the decoder, how do we avoid looking at future tokens (because we are going to generate them and must not cheat)? This is done by "masking," via a mask that assures each token attends only to the previous tokens and itself, but not tokens in the future. The mask can be defined as follows.

$$M(i,j) = \begin{cases} 0 & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases} \tag{17}$$

Recall that scores of each pair of tokens is calculated via $s(i,j) = \frac{\mathbf{q_i}.\mathbf{k_j}}{\sqrt{d_k}}$ in the encoder. Within the decoder, we compute:

$$s^m(i,j) = \frac{\mathbf{q_i}.\mathbf{k_j}}{\sqrt{d_k}} + M(i,j). \tag{18}$$

It can be observed that $-\infty$ is assigned as the score of future tokens and hence, their softmax will be negligible, or practically zero. This assures that these tokens have a weight of zero and are therefore, masked out.

3.2.3 Cross-attention

So far, we have learned that the multi-head self-attention mechanism in the encoder is different from the masked multi-head attention in the decoder, mainly, in that the latter does not look at the future, while the former does so. The encoder and the decoder interact with each other. The output of the encoder is fed to the **second** (non-masked) multi-head attention block in the decoder. Although note that in doing so, the keys and values are drawn from the encoder and the query is drawn from the output of the masked multi-head attention block of the decoder. This is observable from Figure 1 where two arrows going through the (non-masked) multi-head attention block of the decoder come from the encoder. These two arrows are the key and the value. One other arrow goes to the (non-masked) multi-head attention block of the decoder from the masked multi-head attention of the decoder. This procedure is called cross-attention. The algebraic calculations are similar to the calculations in the encoder.

4 Conclusion

The transformer architecture and its variants are widely adopted in everyday practice of machine learning researchers and practitioners, specially in natural language processing. In this article, we explained the transformer architecture in detail and provided numerical examples for it. For this, we showed examples of how algebraic calculations are done in every step of the architecture. We hope that this article helps researchers and practitioners in understanding, debugging, improving, and expanding the architecture.

References

[1] Chen, P.-C., Tsai, H., Bhojanapalli, S., Chung, H. W., Chang, Y.-W., and Ferng, C.-S. A simple and effective positional encoding for transformers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (2021), pp. 2974–2988.

[2] Chu, X., Tian, Z., Zhang, B., Wang, X., and Shen, C. Conditional positional encodings for vision transformers. *arXiv preprint arXiv:2102.10882* (2021).

[3] He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016), pp. 770–778.

[4] Lei Ba, J., Kiros, J. R., and Hinton, G. E. Layer normalization. *ArXiv e-prints* (2016), arXiv–1607.

[5] Shiv, V., and Quirk, C. Novel positional encodings to enable tree-based transformers. *Advances in neural information processing systems 32* (2019).

[6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems* (2017), I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30, Curran Associates, Inc.